

▶ Chapter 13: 유니런 발판 반복 생성과 게임 완성

레트로의 유니티 게임프로그래밍 에센스 (개정판)



이제만 지음

Contents

- Chapter 13 유니런 발판 반복 생성과 게임 완성
 - 13.1 발판 만들기
 - 13.2 발판 생성기 만들기
 - 13.3 빌드하기
 - 13.4 마치며



CHAPTER 13 유니런 발판 반복 생성과 게임 완성

- 발판 구현
- OnEnable() 메서드
- 발판 생성기 구현
- 오브젝트 풀링
- 미리 생성된 발판을 재사용하는 방법

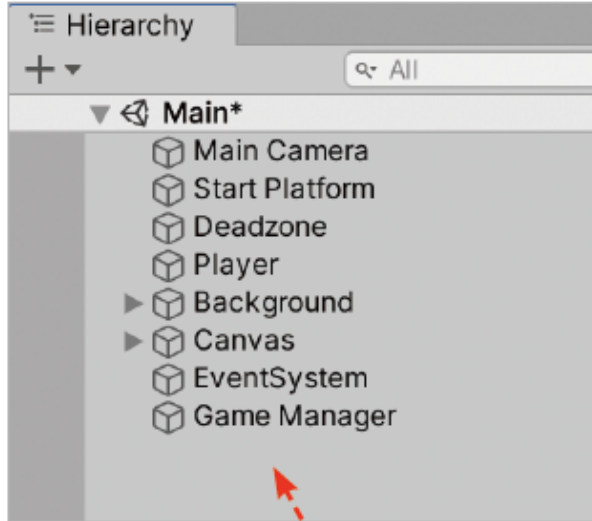
13.1 발판 만들기(1)

- 현재 씬에는 시작 지점의 발판인 Start Platform 게임 오브젝트만 있음
- 시작 지점 이후의 일반 발판은 플레이어 캐릭터가 밟을 경우 점수가 올라가는 기능과 발판 위에 장애물을 생성하는 기능을 가짐
- 발판 프리팹을 편집할 수 있도록 씬에 추가

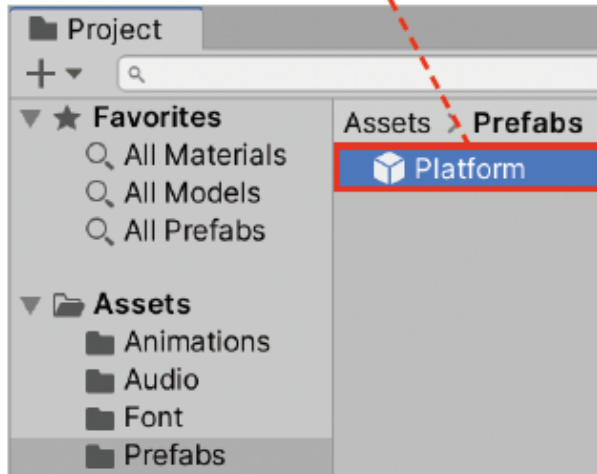
[과정 01] Platform 프리팹 배치하기(1)

- ① Prefabs 폴더에서 Platform 프리팹을 하이어라키 창으로 드래그&드롭
- ② 생성된 Platform 게임 오브젝트의 Sprite Renderer 컴포넌트의 Sorting Layer를 Foreground로 변경

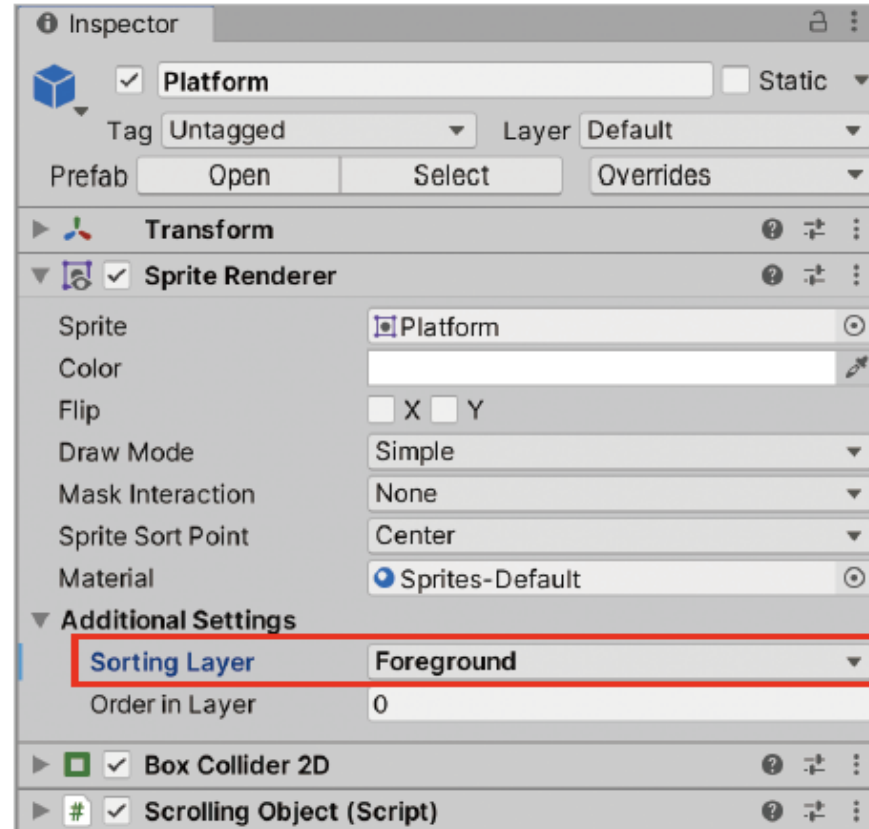
13.1 발판 만들기(2)



① Platform 프리팹을 하이어라키 창으로 드래그&드롭



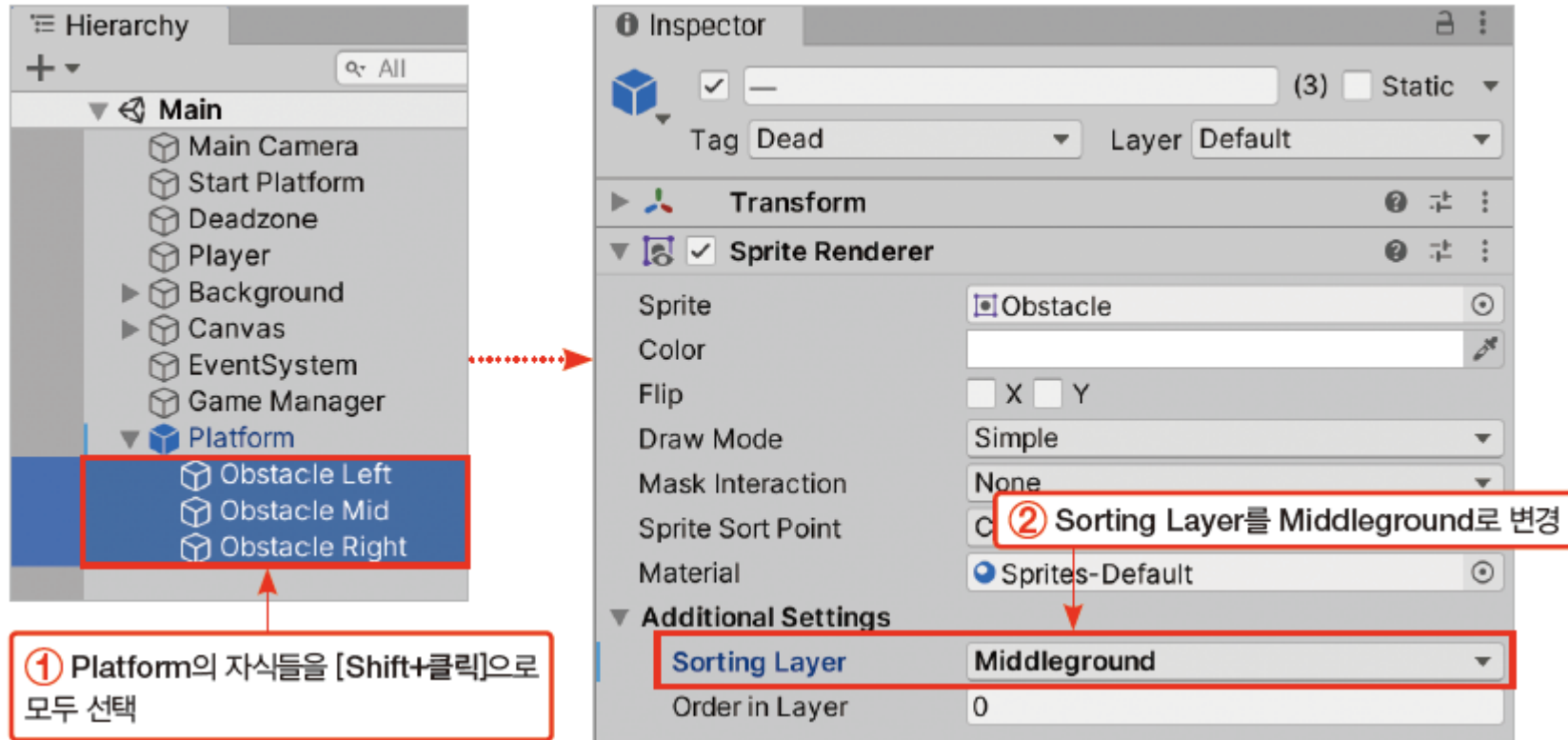
② Platform 게임 오브젝트의 Sorting Layer를 Foreground로 변경



13.1 발판 만들기(3)

[과정 02] Platform 프리팹 배치하기(2)

- ① Platform 게임 오브젝트의 자식들(Obstacle Left, Obstacle Mid, Obstacle Right)을 [Shift+클릭]으로 모두 선택
- ② 자식들의 Sprite Renderer 컴포넌트의 Sorting Layer를 Middleground로 변경



13.1 발판 만들기(4)

13.1.1 Platform 스크립트

- 발판은 플레이어 캐릭터가 자신을 밟으면 게임 점수를 증가
- 자신의 장애물 게임 오브젝트를 일정한 확률로 활성화

[과정 01] Platform 스크립트 적용하기

- ① Scripts 폴더에서 Platform 스크립트를 하이어나키 창의 Platform 게임 오브젝트로 드래그&드롭
- ② Platform 스크립트를 더블 클릭으로 열기

```
using UnityEngine;
```

```
// 발판으로서 필요한 동작을 담은 스크립트
```

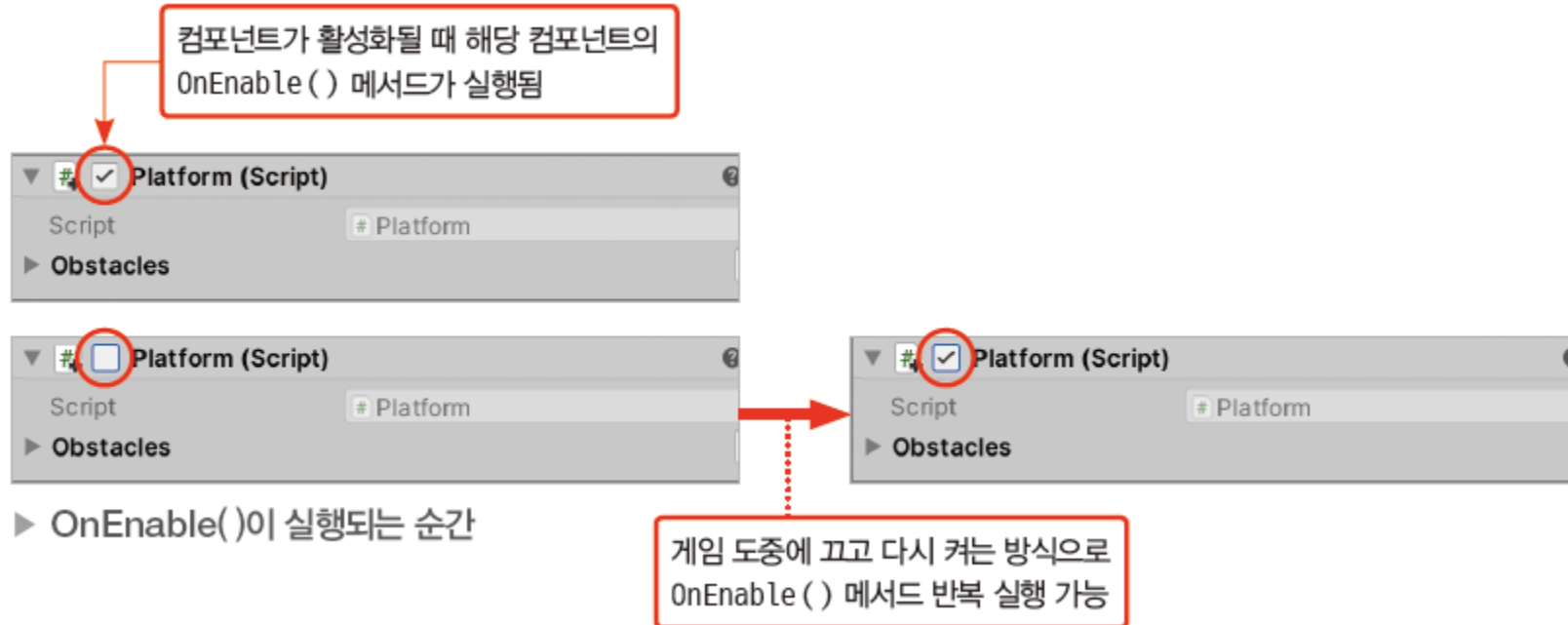
```
public class Platform : MonoBehaviour {  
    public GameObject[] obstacles; // 장애물 오브젝트들  
    private bool stepped = false; // 플레이어 캐릭터가 밟았는가  
  
    // 컴포넌트가 활성화될 때마다 매번 실행되는 메서드  
    private void OnEnable() {  
        // 발판을 리셋하는 처리  
    }  
    void OnCollisionEnter2D(Collision2D collision) {  
        // 플레이어 캐릭터가 자신을 밟았을 때 점수를 추가하는 처리  
    }  
}
```

- ◀ obstacles는 Platform 게임 오브젝트의 자식으로 있는 장애물 게임 오브젝트를 할당할 변수
- ◀ stepped는 플레이어 캐릭터가 자신을 이미 밟았는지 알려주는 상태 변수

13.1 발판 만들기(5)

13.1.2 Platform의 OnEnable() 메서드

- Platform 스크립트의 OnEnable() 메서드를 완성
 - 처음 한 번만 실행되는 Start() 메서드와 달리 OnEnable() 메서드는 컴포넌트가 활성화될 때마다 매번 다시 실행
 - OnEnable() 메서드는 컴포넌트를 끄고 다시 켜는 방식으로 재실행



13.1 발판 만들기(6)

- Platform 스크립트의 OnEnable() 메서드에 발판의 상태를 리셋하는 초기화를 구현

[과정 01] Platform 스크립트의 OnEnable () 완성하기

① Platform 스크립트의 OnEnable() 메서드를 다음과 같이 완성

```
private void OnEnable() {  
    // 밟힘 상태를 리셋  
    stepped = false;  
  
    // 장애물의 수만큼 루프  
    for (int i = 0; i < obstacles.Length; i++)  
    {  
        // 현재 순번의 장애물을 1/3의 확률로 활성화  
        if (Random.Range(0, 3) == 0)  
        {  
            obstacles[i].SetActive(true);  
        }  
        else  
        {  
            obstacles[i].SetActive(false);  
        }  
    }  
}
```

◀ 밟힘 상태 stepped를 false로 초기화

◀ 각각의 장애물 게임 오브젝트를 1/3의 확률로 활성화

◀ obstacles 배열에 있는 장애물 게임 오브젝트들을 순번 i를 사용해
순서대로 접근하고 각각 1/3의 확률로 활성화

13.1 발판 만들기(7)

13.1.3 Platform의 OnCollisionEnter2D() 메서드

- OnCollisionEnter2D() 메서드에 플레이어 캐릭터를 감지하고 점수를 추가하는 코드 작성

[과정 01] Platform 스크립트의 OnCollisionEnter2D () 메서드 완성하기

① Platform 스크립트의 OnCollisionEnter2D() 메서드를 다음과 같이 완성

```
void OnCollisionEnter2D(Collision2D collision) {  
    // 충돌한 상대방의 태그가 Player이고 이전에 플레이어 캐릭터가 밟지 않았다면  
    if (collision.collider.tag == "Player" && !stepped)  
    {  
        // 점수를 추가하고 밟힘 상태를 참으로 변경  
        stepped = true;  
        GameManager.instance.AddScore(1);  
    }  
}
```

13.1 발판 만들기(8)

13.1.4 Platform 스크립트의 전체 코드

```
using UnityEngine;
```

```
// 발판으로서 필요한 동작을 담은 스크립트
```

```
public class Platform : MonoBehaviour {
```

```
    public GameObject[] obstacles; // 장애물 오브젝트들
```

```
    private bool stepped = false; // 플레이어 캐릭터가 밟았는가
```

```
// 컴포넌트가 활성화될 때마다 매번 실행되는 메서드
```

```
private void OnEnable() {
```

```
    // 밟힘 상태를 리셋
```

```
    stepped = false;
```

```
// 장애물의 수만큼 루프
```

```
for (int i = 0; i < obstacles.Length; i++)
```

```
{
```

```
    // 현재 순번의 장애물을 1/3의 확률로 활성화
```

```
    if (Random.Range(0, 3) == 0)
```

```
    {
```

```
        obstacles[i].SetActive(true);
```

```
    }
```

```
else
```

```
{
```

```
    obstacles[i].SetActive(false);
```

```
}
```

```
}
```

```
}
```

```
void OnCollisionEnter2D(Collision2D collision) {
```

```
    // 충돌한 상대방의 태그가 Player이고 이전에 플레이어 캐릭터가 밟지 않았다면
```

```
    if (collision.collider.tag == "Player" && !stepped)
```

```
    {
```

```
        // 점수를 추가하고, 밟힘 상태를 참으로 변경
```

```
        stepped = true;
```

```
        GameManager.instance.AddScore(1);
```

```
    }
```

```
}
```

```
}
```

13.1 발판 만들기(9)

13.1.5 Platform 컴포넌트 설정하기

[과정 01] Platform 컴포넌트 설정하기

- ① 하이어라키 창에서 Platform 게임 오브젝트 선택
- ② Platform 컴포넌트의 Obstacles의 배열 크기를 3으로 변경 > Obstacles 필드 펼치기
- ③ Obstacles에 Obstacle Left, Obstacle Mid, Obstacle Right 할당

① Platform 게임 오브젝트 선택

② Obstacles의 배열 크기를 3으로 변경 > Obstacles 필드 펼치기

③ Obstacles에 Obstacle Left, Obstacle Mid, Obstacle Right 할당

Inspector

Platform

Tag Untagged Layer Default

Prefab Open Select Overrides

Transform

Sprite Renderer

Box Collider 2D

Scrolling Object (Script)

Platform (Script)

Script Platform

Obstacles

Element 0 Obstacle Left

Element 1 Obstacle Mid

Element 2 Obstacle Right

3

Hierarchy

Main*

Main Camera

Start Platform

Deadzone

Player

Background

Canvas

EventSystem

Game Manager

Platform

Obstacle Left

Obstacle Mid

Obstacle Right

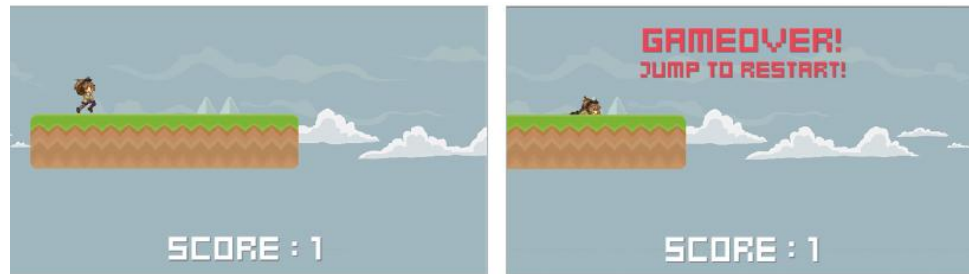
Platform 컴포넌트 설정하기

13.1 발판 만들기(10)

- Platform 게임 오브젝트의 동작 테스트



- ▶ Platform 게임 오브젝트의 동작 테스트하기



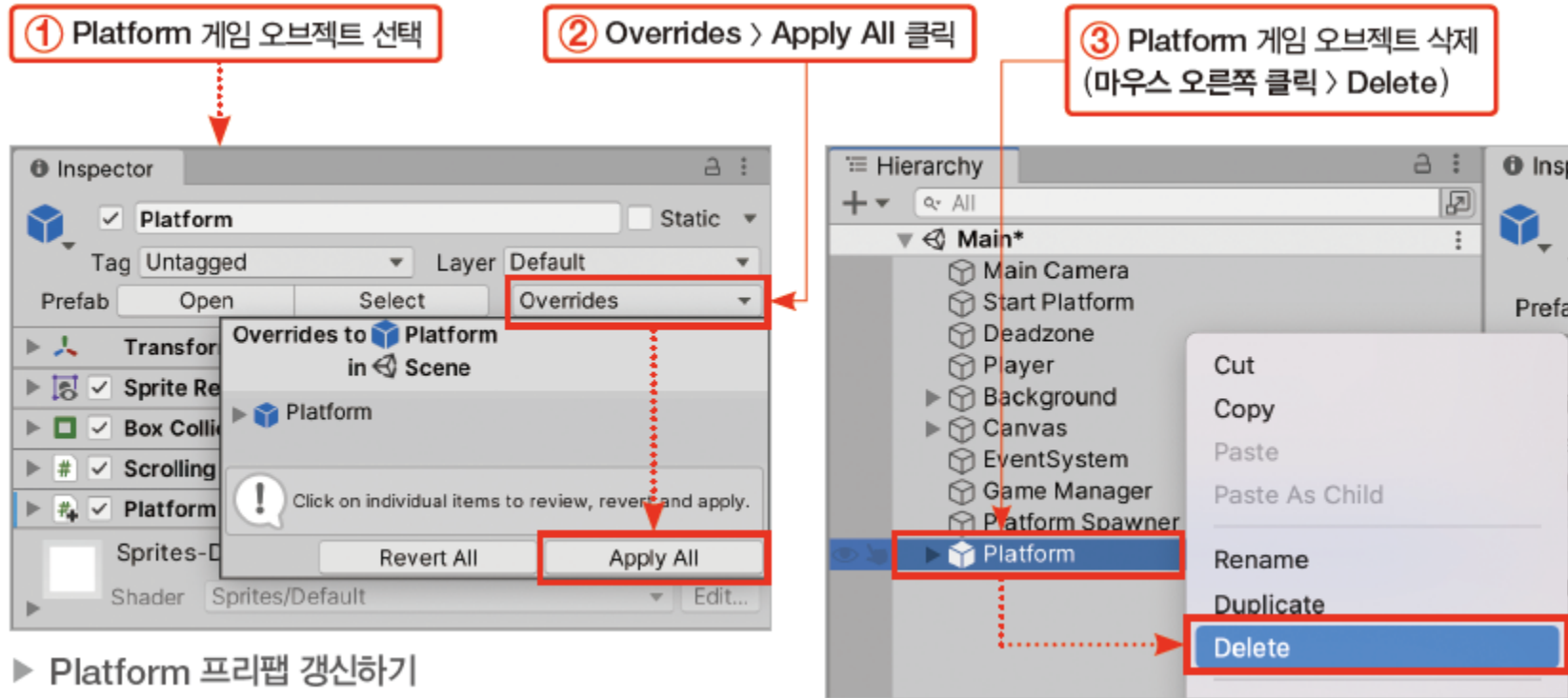
- ▶ 장애물과 부딪혀 사망한 플레이어

13.1 발판 만들기(11)

- Prefabs 폴더의 Platform 프리팹을 씬에서 완성한 Platform 게임 오브젝트로 갱신

[과정 02] Platform 프리팹 갱신하기

- ① 하이어라키 창에서 Platform 게임 오브젝트 선택
- ② 인스펙터 창에서 Overrides > Apply All 클릭
- ③ Platform 게임 오브젝트 삭제(하이어라키 창에서 Platform에 마우스 오른쪽 클릭 > Delete)



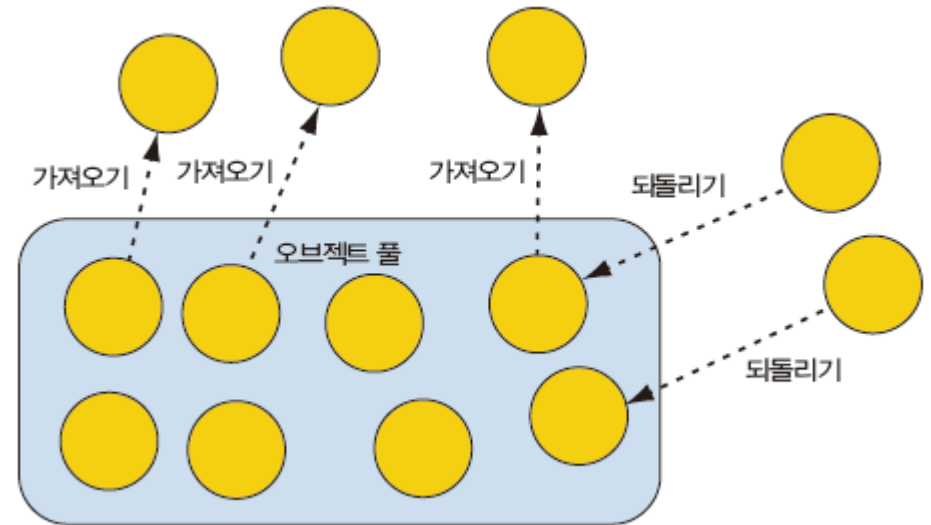
13.2 발판 생성기 만들기(1)

13.2.1 오브젝트 풀링

- 발판 프리팹으로 발판을 무한 반복 생성해서 배치하는 발판 생성기를 구현
- 오브젝트 풀링이란 초기에 필요한 만큼 오브젝트를 미리 만들어 '풀 (웅덩이)'에 쌓아두는 방식



오브젝트 풀링을 사용하지 않는 경우



오브젝트 풀링을 사용하는 경우

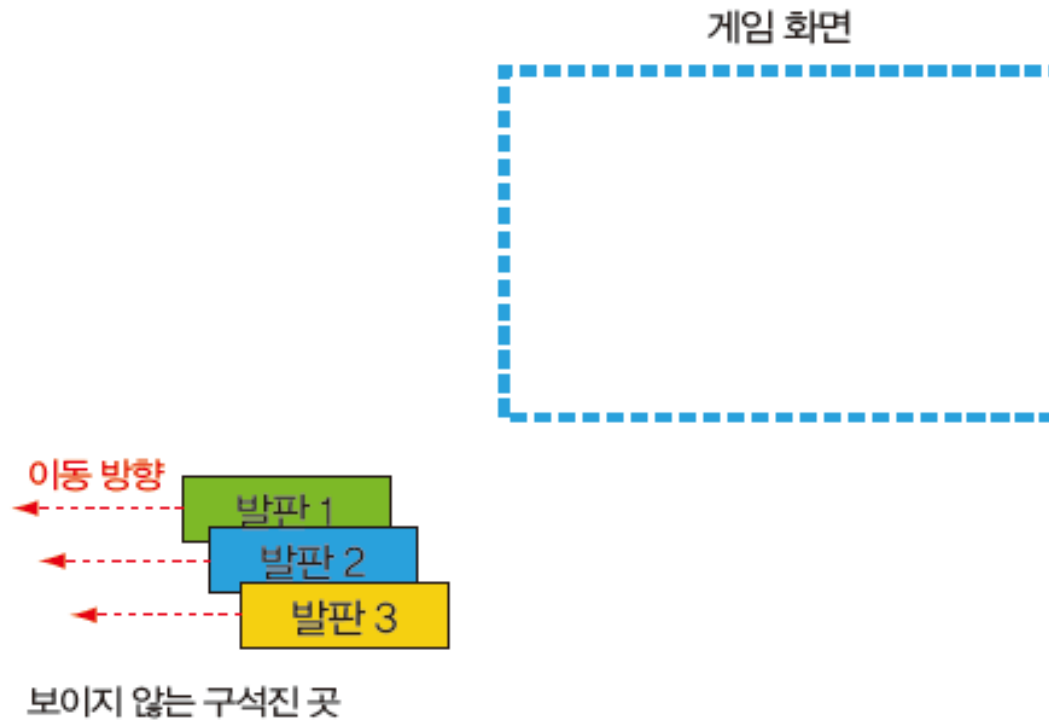
13.2 발판 생성기 만들기(2)

13.2.2 발판 무한 배치 과정

- 오브젝트 풀링의 개념을 사용, 미리 생성한 3개의 발판을 재사용하여 끊임없이 발판이 생성되는 것처럼 보이게 함

[과정 01] 발판 풀 생성

- ① 3개의 발판 오브젝트를 미리 만들어 풀에 배치

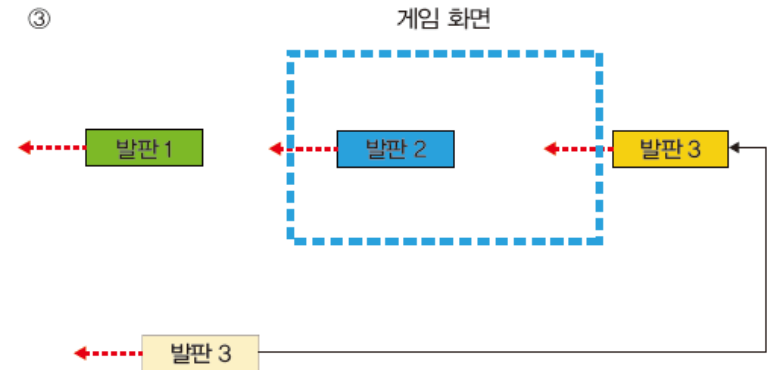
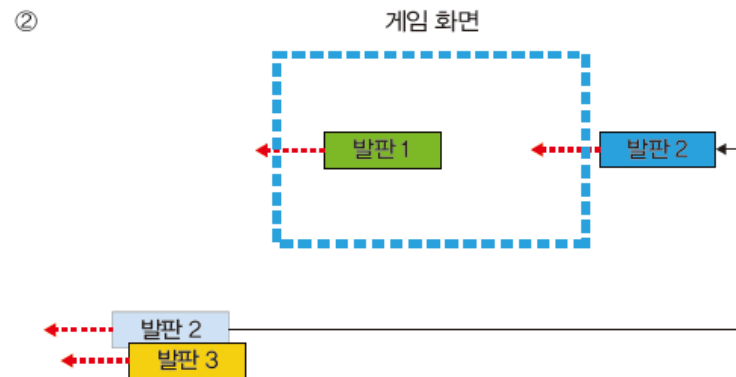
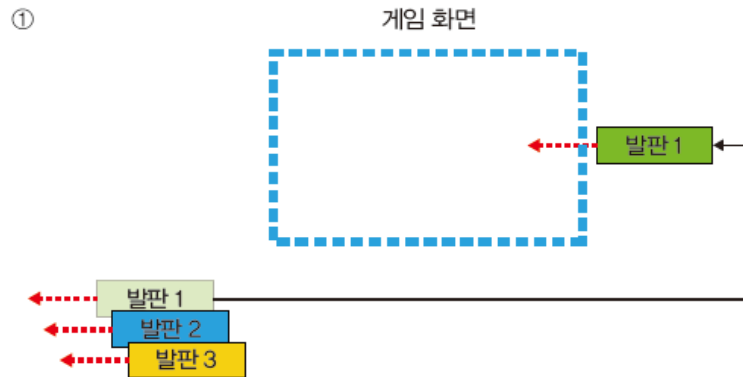


13.2 발판 생성기 만들기(3)

- 게임이 시작되고 일정 시간이 지나면 순서대로 발판 배치를 시작

[과정 02] 첫 번째 배치 루프

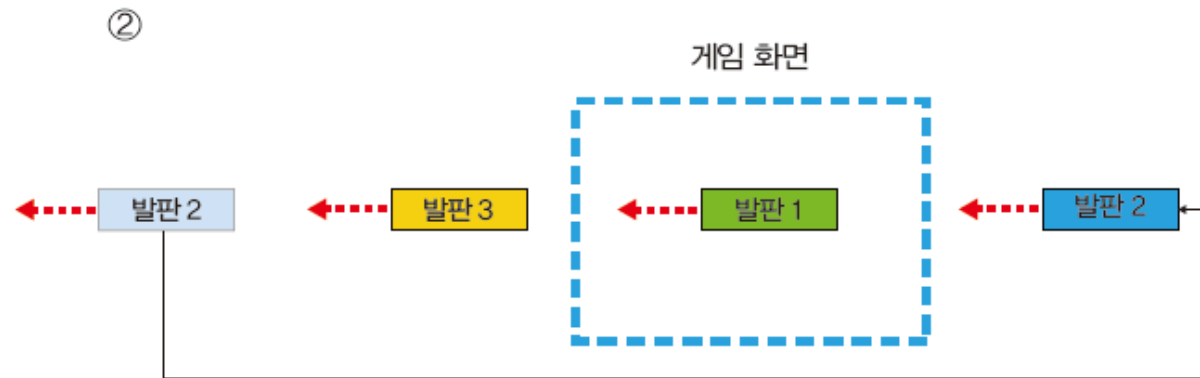
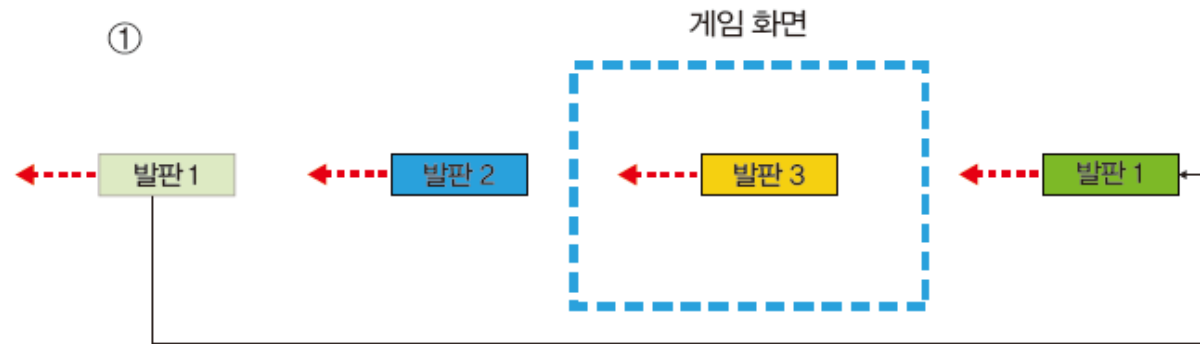
- ① 일정 시간 후 1번 발판을 리셋하고 화면 오른쪽에 재배치
- ② 일정 시간 후 다음(1번 → 2번) 발판을 리셋하고 화면 오른쪽에 재배치
- ③ 일정 시간 후 다음(2번 → 3번) 발판을 리셋하고 화면 오른쪽에 재배치



13.2 발판 생성기 만들기(4)

[과정 03] 첫 번째 이후 배치 루프

- ① 일정 시간 후 다음(3번 → 1번) 발판을 리셋하고 화면 오른쪽에 재배치
- ② 일정 시간 후 다음(1번 → 2번) 발판을 리셋하고 화면 오른쪽에 재배치
- ③ 일정 시간 후 다음(2번 → 3번) 발판을 리셋하고 화면 오른쪽에 재배치
- ④ ...반복



13.2 발판 생성기 만들기(5)

13.2.3 PlatformSpawner 작성하기

- 발판을 생성하고 주기적으로 재배포하는 PlatformSpawner 스크립트를 게임 오브젝트에 추가하고 스크립트 완성

[과정 01] PlatformSpawner 스크립트 열기

- ① 빈 게임 오브젝트 생성(하이어라키 창에서 + > Create Empty)
- ② 생성된 게임 오브젝트의 이름을 Platform Spawner로 변경
- ③ Scripts 폴더의 PlatformSpawner 스크립트를 하이어라키 창의 Platform Spawner 게임 오브젝트로 드래그&드롭
- ④ PlatformSpawner 스크립트를 더블 클릭하여 열기

```
using UnityEngine;

// 발판을 생성하고 주기적으로 재배포하는 스크립트
public class PlatformSpawner : MonoBehaviour {
    public GameObject platformPrefab; // 생성할 발판의 원본 프리팹
    public int count = 3; // 생성할 발판 수

    public float timeBetSpawnMin = 1.25f; // 다음 배치까지의 시간 간격 최솟값
    public float timeBetSpawnMax = 2.25f; // 다음 배치까지의 시간 간격 최댓값
    private float timeBetSpawn; // 다음 배치까지의 시간 간격

    public float yMin = -3.5f; // 배치할 위치의 최소 y 값
    public float yMax = 1.5f; // 배치할 위치의 최대 y 값
    private float xPos = 20f; // 배치할 위치의 x 값
```

```
private GameObject[] platforms; // 미리 생성한 발판들
private int currentIndex = 0; // 사용할 현재 순번의 발판

// 초반에 생성한 발판을 화면 밖에 숨겨둘 위치
private Vector2 poolPosition = new Vector2(0, -25);
private float lastSpawnTime; // 마지막 배치 시점

void Start() {
    // 변수를 초기화하고 사용할 발판을 미리 생성
}

void Update() {
    // 순서를 돌아가며 주기적으로 발판을 배치
}
}
```

13.2 발판 생성기 만들기(6)

13.2.4 PlatformSpawner의 필드

- platformPrefab에 복제 생성할 발판 게임 오브젝트의 원본 프리팹이 할당
 - count는 복제 생성할 발판 수를 결정

```
public GameObject platformPrefab;  
public int count = 3;
```

- timeBetSpawn은 현재 순번의 발판이 배치된 후 다음번 발판을 배치할 때까지 소요되는 시간

```
public float timeBetSpawnMin = 1.25f;  
public float timeBetSpawnMax = 2.25f;  
private float timeBetSpawn;
```

- 발판을 재배치할 위치를 결정

```
public float yMin = -3.5f;  
public float yMax = 1.5f;  
private float xPos = 20f;
```

13.2 발판 생성기 만들기(7)

- 현재 사용할 발판의 순번을 가리킬 변수

```
private GameObject[] platforms;  
private int currentIndex = 0;
```

- 발판 게임 오브젝트를 초반에 모아둘 장소와 마지막으로 재배치를 실행한 시점을 저장하는 변수

```
private Vector2 poolPosition = new Vector2(0, -25);  
private float lastSpawnTime;
```

13.2 발판 생성기 만들기(8)

13.2.5 PlatformSpawner의 Start() 메서드

[과정 01] PlatformSpawner의 Start () 메서드 완성하기

① PlatformSpawner의 Start() 메서드를 다음과 같이 완성

```
void Start() {  
    // count만큼의 공간을 가지는 새로운 발판 배열 생성  
    platforms = new GameObject[count];  
  
    // count만큼 루프하면서 발판 생성  
    for (int i = 0; i < count; i++)  
    {  
        // platformPrefab을 원본으로 새 발판을 poolPosition 위치에 복제 생성  
        // 생성된 발판을 platform 배열에 할당  
        platforms[i] = Instantiate(platformPrefab, poolPosition, Quaternion.identity);  
    }  
  
    // 마지막 배치 시점 초기화  
    lastSpawnTime = 0f;  
    // 다음번 배치까지의 시간 간격을 0으로 초기화  
    timeBetSpawn = 0f;  
}
```

13.2 발판 생성기 만들기(9)

- 발판 게임 오브젝트의 수 count만큼의 공간을 가지는 GameObject 배열을 생성하여 platforms에 할당

```
platforms = new GameObject[count];
```

- for 문으로 count만큼 루프하면서 발판 게임 오브젝트를 생성하여 배열의 각 원소에 순서대로 할당

```
for (int i = 0; i < count; i++)  
{  
    platforms[i] = Instantiate(platformPrefab, poolPosition, Quaternion.identity);  
}
```

- 복제 생성한 발판 게임 오브젝트를 0번째부터 count-1번째까지 i를 사용해 platforms의 각 원소에 하나씩 할당

```
platforms[i] = Instantiate (platformPrefab, poolPosition, Quaternion.identity);
```

- 가장 최근의 재배포 시점을 나타내는 lastSpawnTime과 다음번 배치까지의 시간 간격인 timeBetSpawn의 값을 0으로 초기화

```
lastSpawnTime = 0f;  
timeBetSpawn = 0f;
```

13.2 발판 생성기 만들기(10)

13.2.6 PlatformSpanwer의 Update() 메서드

- Start() 메서드에서 만든 발판 게임 오브젝트를 돌아가며 사용하는 방식으로 발판 무한 배치를 구현

[과정 01] PlatformSpawner 스크립트의 Update () 메서드 완성하기

① PlatformSpawner 스크립트의 Update() 메서드를 다음과 같이 완성

```
void Update() {  
    // 게임오버 상태에서는 동작하지 않음  
    if (GameManager.instance.isGameOver)  
    {  
        return;  
    }  
  
    // 마지막 배치 시점에서 timeBetSpawn 이상 시간이 흘렀다면  
    if (Time.time >= lastSpawnTime + timeBetSpawn)  
    {  
        // 기록된 마지막 배치 시점을 현재 시점으로 갱신  
        lastSpawnTime = Time.time;  
  
        // 다음 배치까지의 시간 간격을 timeBetSpawnMin, timeBetSpawnMax 사이에서 랜덤 설정  
        timeBetSpawn = Random.Range(timeBetSpawnMin, timeBetSpawnMax);  
  
        // 배치할 위치의 높이를 yMin과 yMax 사이에서 랜덤 설정  
        float yPos = Random.Range(yMin, yMax);
```

```
        // 사용할 현재 순번의 발판 게임 오브젝트를 비활성화하고 즉시 다시 활성화  
        // 이때 발판의 Platform 컴포넌트의 OnEnable 메서드가 실행될  
        platforms[currentIndex].SetActive(false);  
        platforms[currentIndex].SetActive(true);  
  
        // 현재 순번의 발판을 화면 오른쪽에 재배치  
        platforms[currentIndex].transform.position = new Vector2(xPos, yPos);  
        // 순번 넘기기  
        currentIndex++;  
  
        // 마지막 순번에 도달했다면 순번을 리셋  
        if (currentIndex >= count)  
        {  
            currentIndex = 0;  
        }  
    }  
}
```

13.2 발판 생성기 만들기(11)

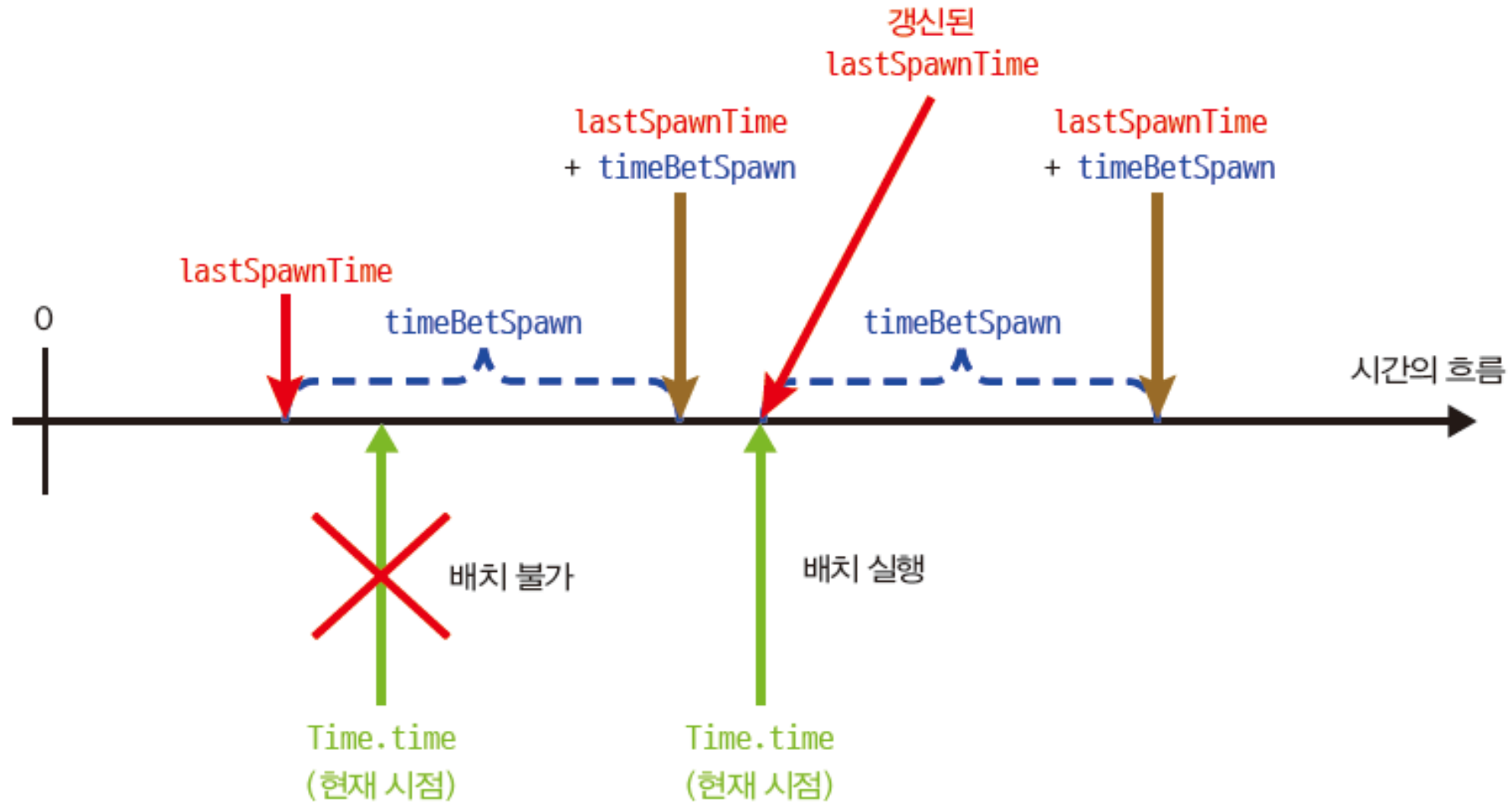
- if 문으로 게임오버 상태에서는 더 이상 처리가 진행되지 않도록 차단

```
if (GameManager.instance.isGameOver)
{
    return;
}
```

- 발판을 재배치한 가장 최근 시점에서 지금까지 경과 시간 확인
 - $\text{Time.time} \geq \text{lastSpawnTime} + \text{timeBetSpawn}$ 은 현재 시점(Time.time)이 다음번 배치가 예약된 시점($\text{lastSpawnTime} + \text{timeBetSpawn}$)과 같거나 그 이상의 시간이 지났는지 검사

```
if (Time.time >= lastSpawnTime + timeBetSpawn)
```

13.2 발판 생성기 만들기(12)



▶ 재배치 시점을 표현한 타임라인

13.2 발판 생성기 만들기(13)

- 해당 조건을 만족하면 if 문 블록이 실행되어 가장 최근 배치 시점을 갱신하고 발판을 실제로 배치

```
lastSpawnTime = Time.time;  
timeBetSpawn = Random.Range(timeBetSpawnMin, timeBetSpawnMax);
```

- 발판을 재배치할 높이를 랜덤 값으로 설정

```
float yPos = Random.Range(yMin, yMax);
```

- 재배치할 현재 순번의 발판 게임 오브젝트를 비활성화하고 즉시 다시 활성화

```
platforms[currentIndex].SetActive(false);  
platforms[currentIndex].SetActive(true);
```

13.2 발판 생성기 만들기(14)

- 재배치할 발판 게임 오브젝트를 리셋했다면 위치를 변경하고 순번을 넘김

```
platforms[currentIndex].transform.position = new Vector2(xPos, yPos);  
currentIndex++;
```

- 순번의 마지막에 도달하면 순번을 리셋

```
if (currentIndex >= count)  
{  
    currentIndex = 0;  
}
```

13.2 발판 생성기 만들기(15)

13.2.7 PlatformSpawner 스크립트의 전체 코드

```
using UnityEngine;

// 발판을 생성하고 주기적으로 재배포하는 스크립트
public class PlatformSpawner : MonoBehaviour {
    public GameObject platformPrefab; // 생성할 발판의 원본 프리팹
    public int count = 3; // 생성할 발판 수

    public float timeBetSpawnMin = 1.25f; // 다음 배치까지의 시간 간격 최소값
    public float timeBetSpawnMax = 2.25f; // 다음 배치까지의 시간 간격 최대값
    private float timeBetSpawn; // 다음 배치까지의 시간 간격

    public float yMin = -3.5f; // 배치할 위치의 최소 y 값
    public float yMax = 1.5f; // 배치할 위치의 최대 y 값
    private float xPos = 20f; // 배치할 위치의 x 값

    private GameObject[] platforms; // 미리 생성한 발판들
    private int currentIndex = 0; // 사용할 현재 순번의 발판
```

▶ 다음 쪽에 이어짐

13.2 발판 생성기 만들기(16)

◀ 앞쪽에 이어

// 초반에 생성한 발판을 화면 밖에 숨겨둘 위치

```
private Vector2 poolPosition = new Vector2(0, -25);
```

```
private float lastSpawnTime; // 마지막 배치 시점
```

// 변수를 초기화하고 사용할 발판을 미리 생성

```
void Start() {
```

// count만큼의 공간을 가지는 새로운 발판 배열 생성

```
platforms = new GameObject[count];
```

// count만큼 루프하면서 발판 생성

```
for (int i = 0; i < count; i++)
```

```
{
```

// platformPrefab을 원본으로 새 발판을 poolPosition 위치에 복제 생성

// 생성된 발판을 platform 배열에 할당

```
platforms[i] = Instantiate(platformPrefab, poolPosition, Quaternion.identity);
```

```
}
```

// 마지막 배치 시점 초기화

```
lastSpawnTime = 0f;
```

▶ 다음 쪽에 이어짐

13.2 발판 생성기 만들기(17)

◀ 앞쪽에 이어

```
// 다음번 배치까지의 시간 간격을 0으로 초기화
timeBetSpawn = 0f;
}

void Update() {
    // 게임오버 상태에서는 동작하지 않음
    if (GameManager.instance.isGameOver)
    {
        return;
    }

    // 마지막 배치 시점에서 timeBetSpawn 이상 시간이 흘렀다면
    if (Time.time >= lastSpawnTime + timeBetSpawn)
    {
        // 기록된 마지막 배치 시점을 현재 시점으로 갱신
        lastSpawnTime = Time.time;

        // 다음 배치까지의 시간 간격을 timeBetSpawnMin, timeBetSpawnMax 사이에서 랜덤 설정
        timeBetSpawn = Random.Range(timeBetSpawnMin, timeBetSpawnMax);

        // 배치할 위치의 높이를 yMin과 yMax 사이에서 랜덤 설정
        float yPos = Random.Range(yMin, yMax);
```

▶ 다음 쪽에 이어짐

13.2 발판 생성기 만들기(18)

◀ 앞쪽에 이어

```
// 사용할 현재 순번의 발판 게임 오브젝트를 비활성화하고 즉시 다시 활성화
// 이때 발판의 Platform 컴포넌트의 OnEnable 메서드가 실행됨
platforms[currentIndex].SetActive(false);
platforms[currentIndex].SetActive(true);

// 현재 순번의 발판을 화면 오른쪽에 재배치
platforms[currentIndex].transform.position = new Vector2(xPos, yPos);
// 순번 넘기기
currentIndex++;

// 마지막 순번에 도달했다면 순번을 리셋
if (currentIndex >= count)
{
    currentIndex = 0;
}
}
}
}
```

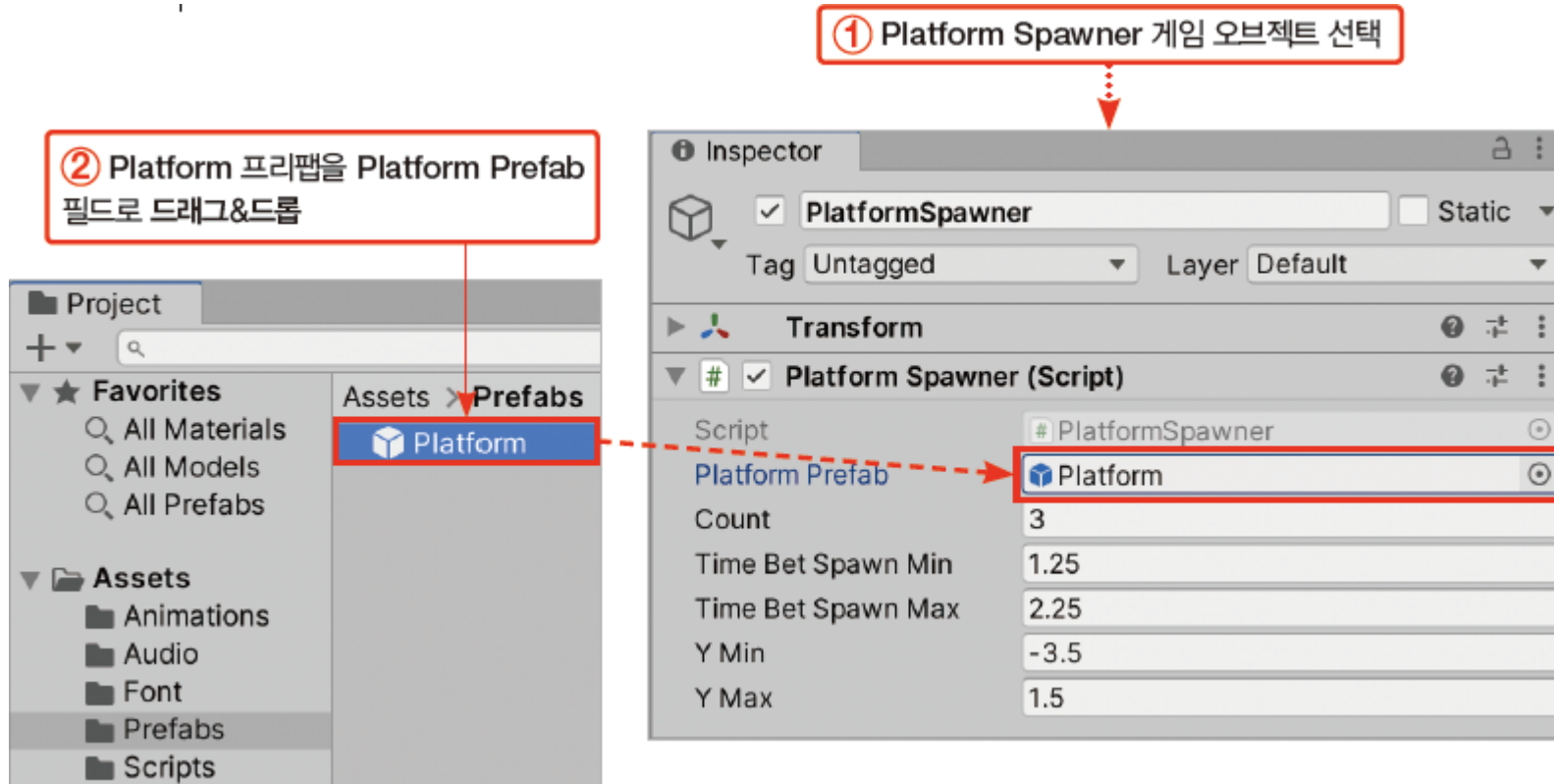
13.2 발판 생성기 만들기(19)

13.2.8 PlatformSpawner 컴포넌트 설정하기

- 완성한 PlatformSpawner 컴포넌트를 설정

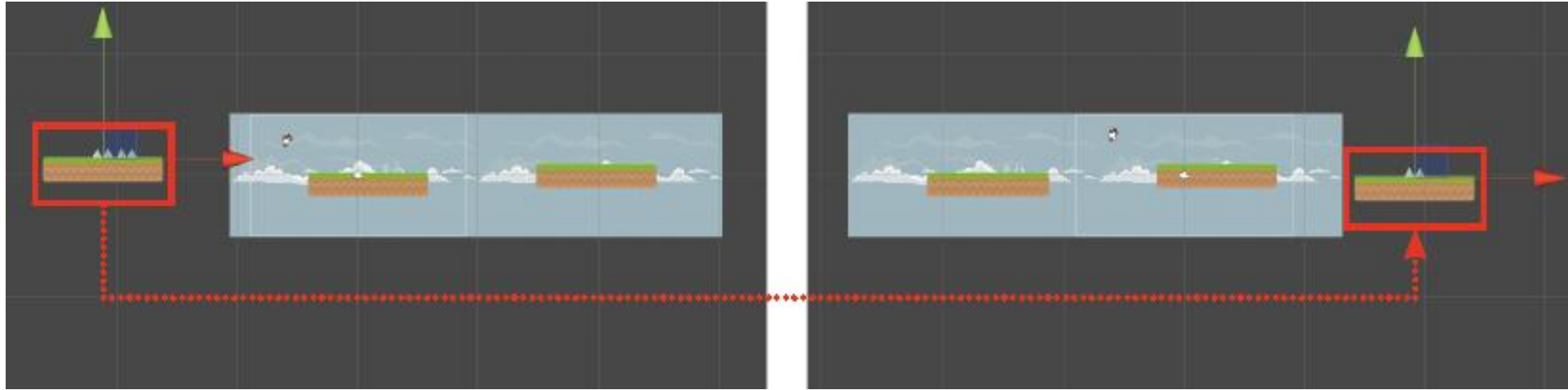
[과정 01] PlatformSpawner 컴포넌트 설정하기

- ① 하이어라키 창에서 Platform Spawner 게임 오브젝트 선택
- ② Prefabs 폴더의 Platform 프리팹을 PlatformSpawner 컴포넌트의 Platform Prefab 필드로 드래그&드롭



13.2 발판 생성기 만들기(20)

- 완성된 발판 생성기 테스트



▶ 발판 생성기와 발판의 동작

13.3 빌드하기(1)

- 완성한 게임의 주요 기능
 - 플레이어 캐릭터는 상황에 따라 뛰고, 점프하고, 사망하는 애니메이션을 재생
 - 게임이 시작되면 배경이 반복되고 새로운 발판이 주기적으로 게임 화면 오른쪽에 배치
 - 플레이어 캐릭터가 다가오는 발판을 향해 점프하고 무사히 착지할 때마다 점수가 1점씩 증가
 - 증가한 점수는 UI로 확인

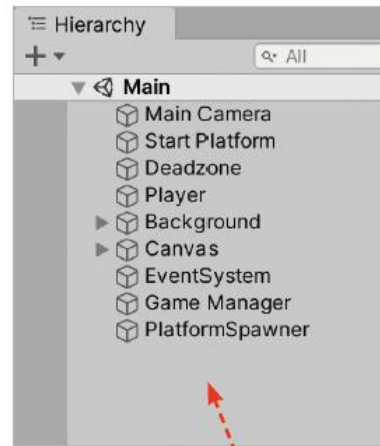


13.3 빌드하기(2)

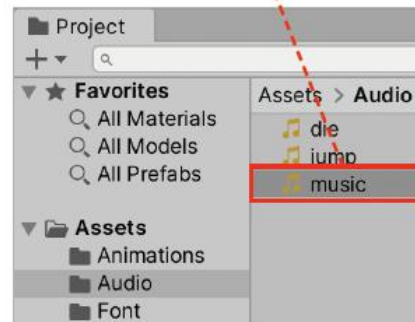
13.3.1 배경음악 추가하기

[과정 01] 배경음악 추가하기

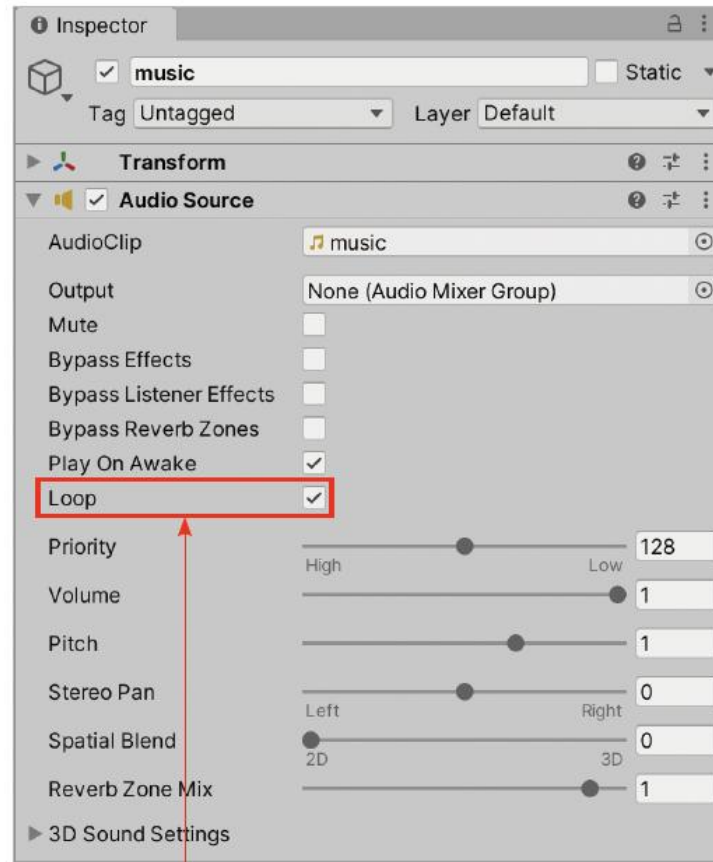
- ① 프로젝트 창의 Audio 폴더에서 music 오디오 클립을 하이어라키 창으로 드래그&드롭
- ② 생성된 music 게임 오브젝트의 Audio Source 컴포넌트의 Loop 체크



① music 오디오 클립을 하이어라키 창으로 드래그&드롭



▶ 배경음악 추가하기



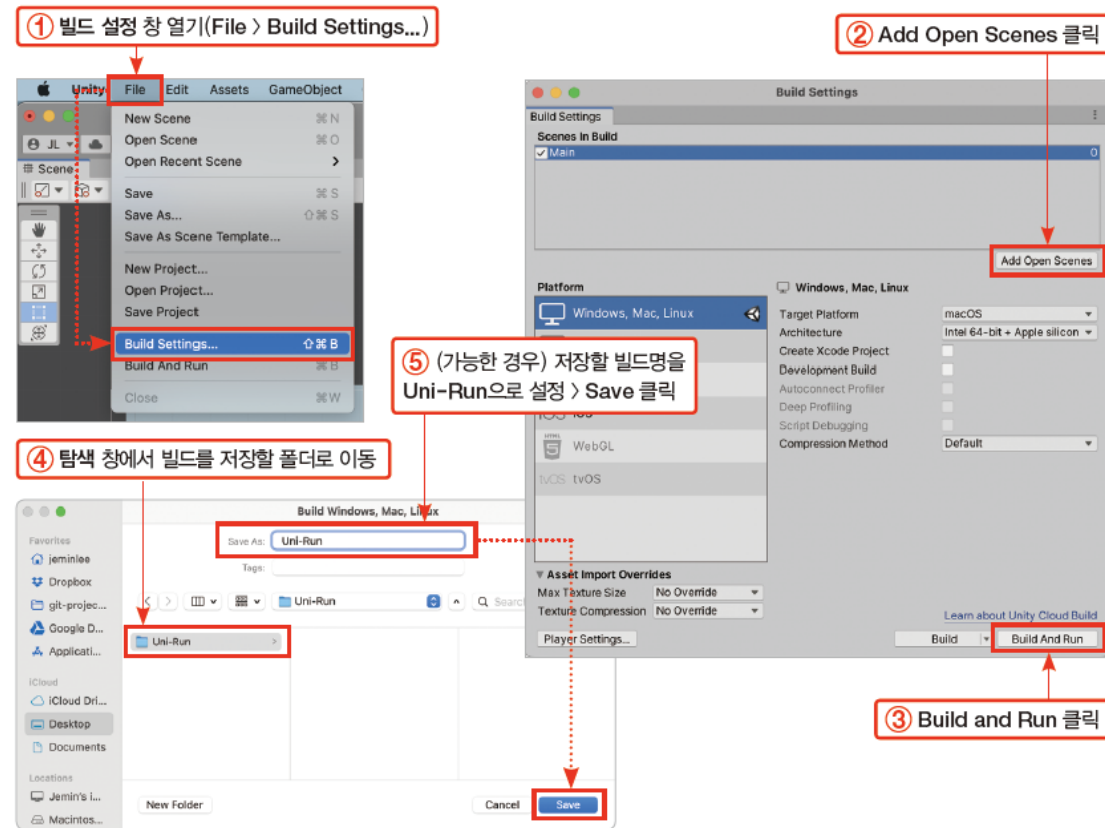
② music 게임 오브젝트의 Audio Source 컴포넌트의 Loop 체크

13.3 빌드하기(3)

13.3.2 유니런 빌드하기

[과정 01] 유니런 빌드하기

- ① 빌드 설정 창 열기(상단 메뉴의 File > Build Settings...)
- ② 빌드 설정 창에서 Add Open Scenes 클릭
- ③ Build and Run 클릭
- ④ 탐색 창에서 빌드를 저장할 폴더로 이동
- ⑤ (가능한 경우) 저장할 빌드명을 Uni-Run으로 설정 > Save 클릭



13.3 빌드하기(4)

- 빌드가 끝나면 실행 파일이 지정한 폴더에 저장
- 유니티 런처가 실행되며, Play 버튼을 눌러 게임을 플레이



13.4 마치며

이 장에서 배운 내용 요약

- OnEnable() 메서드는 컴포넌트가 활성화될 때마다 매번 실행
- 오브젝트 풀링은 필요한 오브젝트를 미리 생성하여 사용하는 방식
- 오브젝트 풀링에서는 오브젝트를 실시간 생성하지 않고 풀에 있는 오브젝트를 가져다 사용
- 오브젝트 풀링에서는 오브젝트를 실시간으로 파괴하지 않고 파괴할 오브젝트를 풀로 반환
- 오브젝트 풀링을 사용하면 초기 로딩은 길어지지만 실시간 생성/파괴에 의한 성능 저하를 차단
- 오디오 소스 컴포넌트의 Loop를 체크하면 오디오가 반복 재생